Shadowfox – Cybersecurity Internship Report

Name: Aashif M
Email: Aashif8056@gmail.com
Domain: Cybersecurity
Role: Internship Trainee
Duration: 1 Month

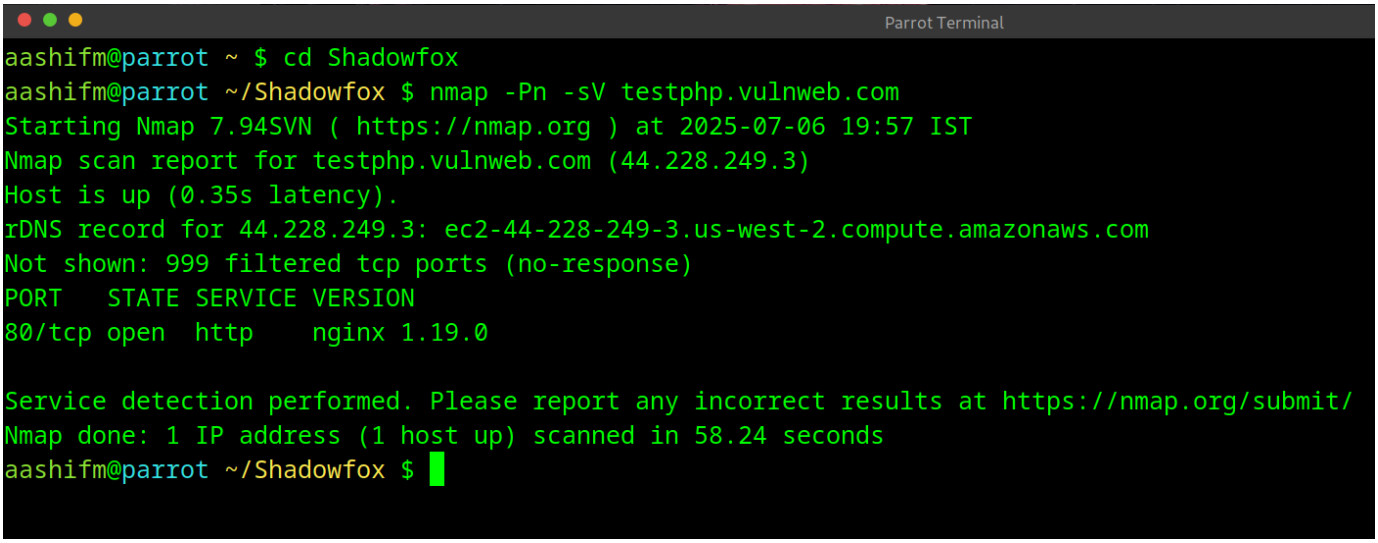
Overview

During my internship at Shadowfox in the Cybersecurity domain, I participated in a range of practical penetration testing and security assessment tasks. This report documents my methodologies, tools used, and the key outcomes achieved at each skill level: Easy, Intermediate, and Hard.

Easy Level Tasks

1. Open Port and Service Discovery

- Objective: Identify open ports and running services of the test web application.
- Target: http://testphp.vulnweb.com/
- Tool Used: Nmap
  - nmap -Pn -sV http://testphp.vulnweb.com/



- Findings:
  - Discovered various open ports providing insight into available services and potential attack surfaces such as HTTP, FTP, and SSH. The use of service version detection enabled identification of underlying software, which is crucial for vulnerability assessment.

2. Directory Brute-forcing

- Objective: Enumerate all accessible directories in the web application to uncover hidden resources or endpoints.
- Tool Used: Gobuster
  - gobuster dir -u http://testphp.vulnweb.com/ -w /usr/share/wordlists/dirb/common.txt
    - Utilizes a popular wordlist to maximize coverage and accuracy.

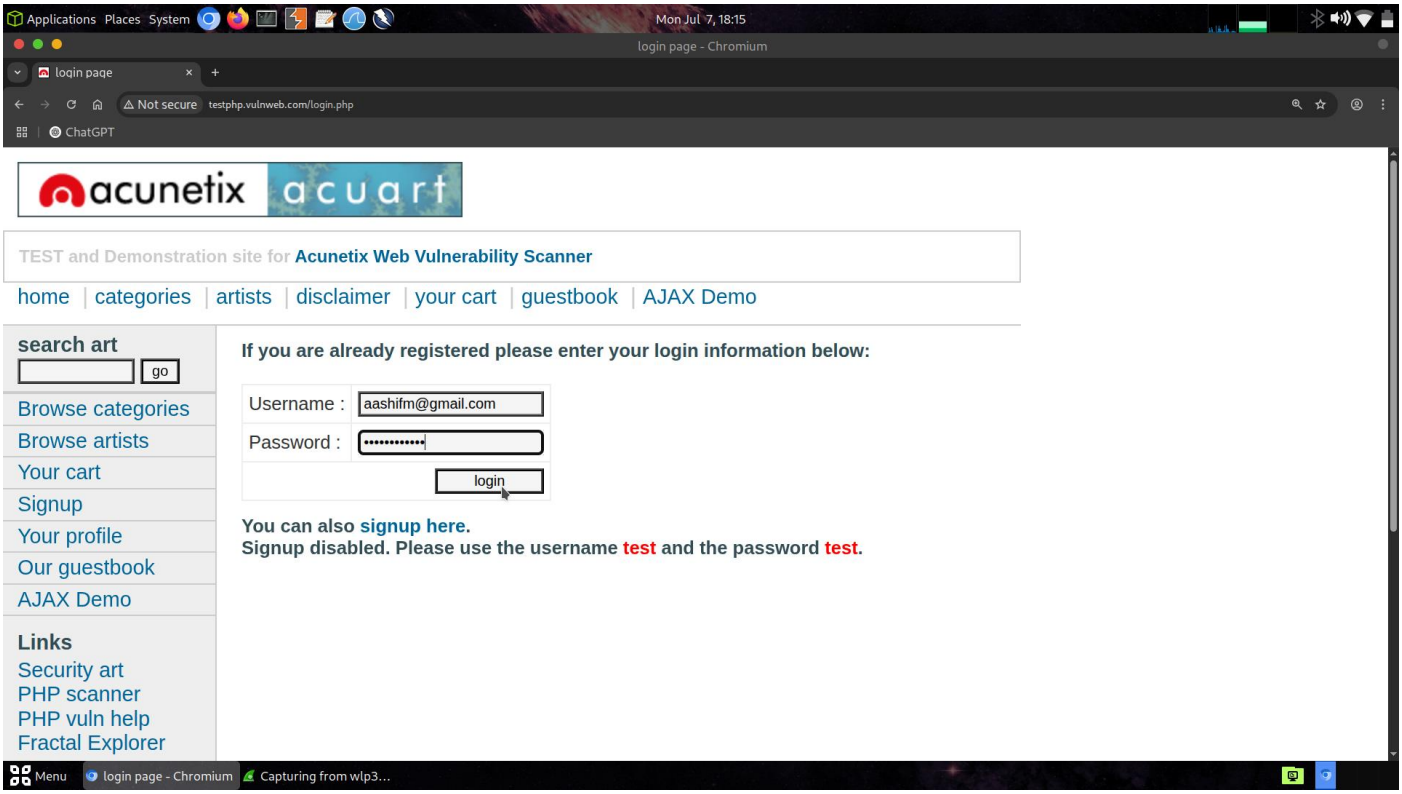
```
/ip                   (Status: 500) [Size: 177]
/katalog              (Status: 500) [Size: 177]
/lat_driver           (Status: 500) [Size: 177]
/learners             (Status: 500) [Size: 177]
/lib                  (Status: 500) [Size: 177]
/links_submit         (Status: 500) [Size: 177]
/magazine             (Status: 500) [Size: 177]
/migrate              (Status: 500) [Size: 177]
/mobile               (Status: 500) [Size: 177]
/mount                (Status: 500) [Size: 177]
/mt-search            (Status: 500) [Size: 177]
/myfaces              (Status: 500) [Size: 177]
/objects              (Status: 500) [Size: 177]
/of                   (Status: 500) [Size: 177]
/offerdetail          (Status: 500) [Size: 177]
/ogl                  (Status: 500) [Size: 177]
/oldie                (Status: 500) [Size: 177]
/opencart             (Status: 500) [Size: 177]
/package              (Status: 500) [Size: 177]
/payment_gateway      (Status: 500) [Size: 177]
/pictures             (Status: 301) [Size: 169] [--> http://testphp.vulnweb.com/pictures/]
/port                 (Status: 500) [Size: 177]
/Press                (Status: 500) [Size: 177]
/prepare              (Status: 500) [Size: 177]
/secured              (Status: 301) [Size: 169] [--> http://testphp.vulnweb.com/secured/]
/system-administration (Status: 500) [Size: 177]
/tabs                 (Status: 500) [Size: 177]
/tagline              (Status: 500) [Size: 177]
/toolbar              (Status: 500) [Size: 177]
/user                 (Status: 500) [Size: 177]
/usa                  (Status: 500) [Size: 177]
/vbseo                (Status: 500) [Size: 177]
/vbscripts            (Status: 500) [Size: 177]
/vehicle              (Status: 500) [Size: 177]
/vendor               (Status: 301) [Size: 169] [--> http://testphp.vulnweb.com/vendor/]
/W3SVC3               (Status: 500) [Size: 177]
/web1                 (Status: 500) [Size: 177]
/webtrends            (Status: 500) [Size: 177]
/workflowtasks        (Status: 500) [Size: 177]
/xalan                (Status: 500) [Size: 177]
Progress: 4614 / 4615 (99.98%)
===============================================================
Finished
===============================================================
aashifm@parrot ~/Shadowfox $
```


```
/ip                   (Status: 500) [Size: 177]
/katalog              (Status: 500) [Size: 177]
/lat_driver           (Status: 500) [Size: 177]
/learners             (Status: 500) [Size: 177]
/lib                  (Status: 500) [Size: 177]
/links_submit         (Status: 500) [Size: 177]
/magazine             (Status: 500) [Size: 177]
/migrate              (Status: 500) [Size: 177]
/mobile               (Status: 500) [Size: 177]
/mount                (Status: 500) [Size: 177]
/mt-search            (Status: 500) [Size: 177]
/myfaces              (Status: 500) [Size: 177]
/objects              (Status: 500) [Size: 177]
/of                   (Status: 500) [Size: 177]
/offerdetail          (Status: 500) [Size: 177]
/ogl                  (Status: 500) [Size: 177]
/oldie                (Status: 500) [Size: 177]
/opencart             (Status: 500) [Size: 177]
/package              (Status: 500) [Size: 177]
/payment_gateway      (Status: 500) [Size: 177]
/pictures             (Status: 301) [Size: 169] [--> http://testphp.vulnweb.com/pictures/]
/port                 (Status: 500) [Size: 177]
/Press                (Status: 500) [Size: 177]
/prepare              (Status: 500) [Size: 177]
/secured              (Status: 301) [Size: 169] [--> http://testphp.vulnweb.com/secured/]
/system-administration (Status: 500) [Size: 177]
/tabs                 (Status: 500) [Size: 177]
/tagline              (Status: 500) [Size: 177]
/toolbar              (Status: 500) [Size: 177]
/user                 (Status: 500) [Size: 177]
/usa                  (Status: 500) [Size: 177]
/vbseo                (Status: 500) [Size: 177]
/vbscripts            (Status: 500) [Size: 177]
/vehicle              (Status: 500) [Size: 177]
/vendor               (Status: 301) [Size: 169] [--> http://testphp.vulnweb.com/vendor/]
/W3SVC3               (Status: 500) [Size: 177]
/web1                 (Status: 500) [Size: 177]
/webtrends            (Status: 500) [Size: 177]
/workflowtasks        (Status: 500) [Size: 177]
/xalan                (Status: 500) [Size: 177]
Progress: 4614 / 4615 (99.98%)
===============================================================
Finished
===============================================================
aashifm@parrot ~/Shadowfox $
```

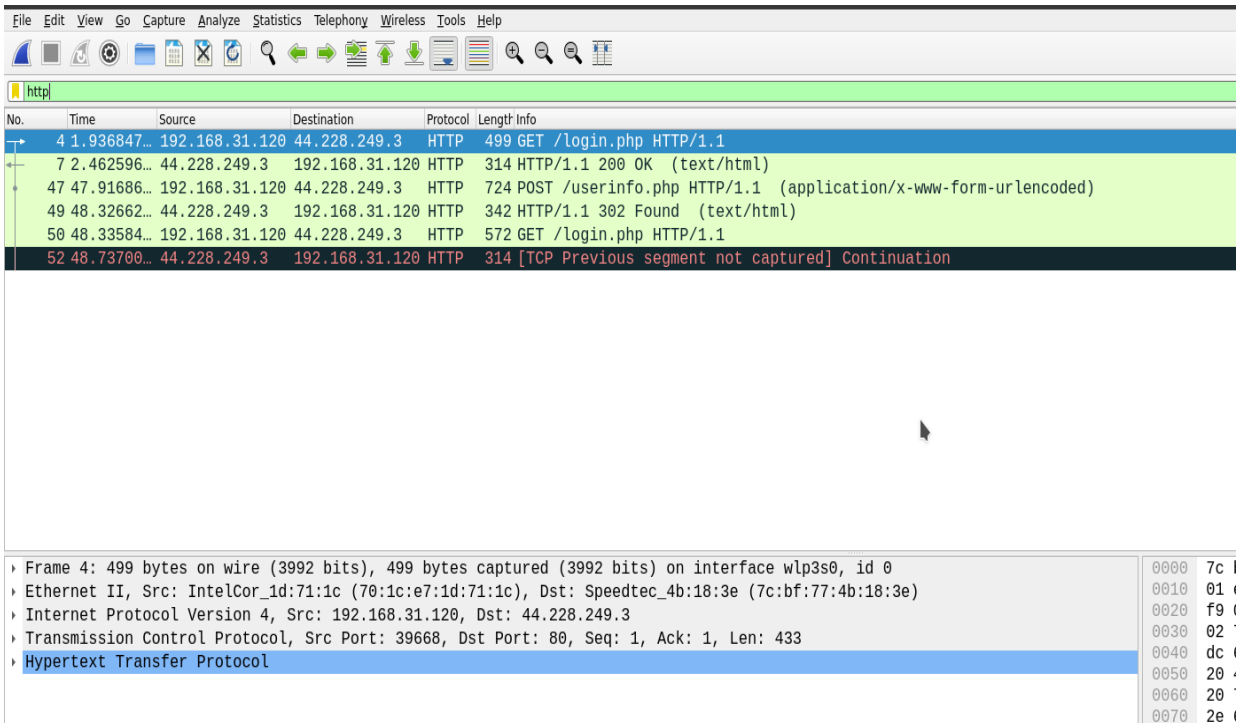

- Findings:
  - Successfully identified multiple directories, supporting deeper penetration testing efforts and improving awareness of the app's structure.

3. Credential Interception via Network Traffic Analysis

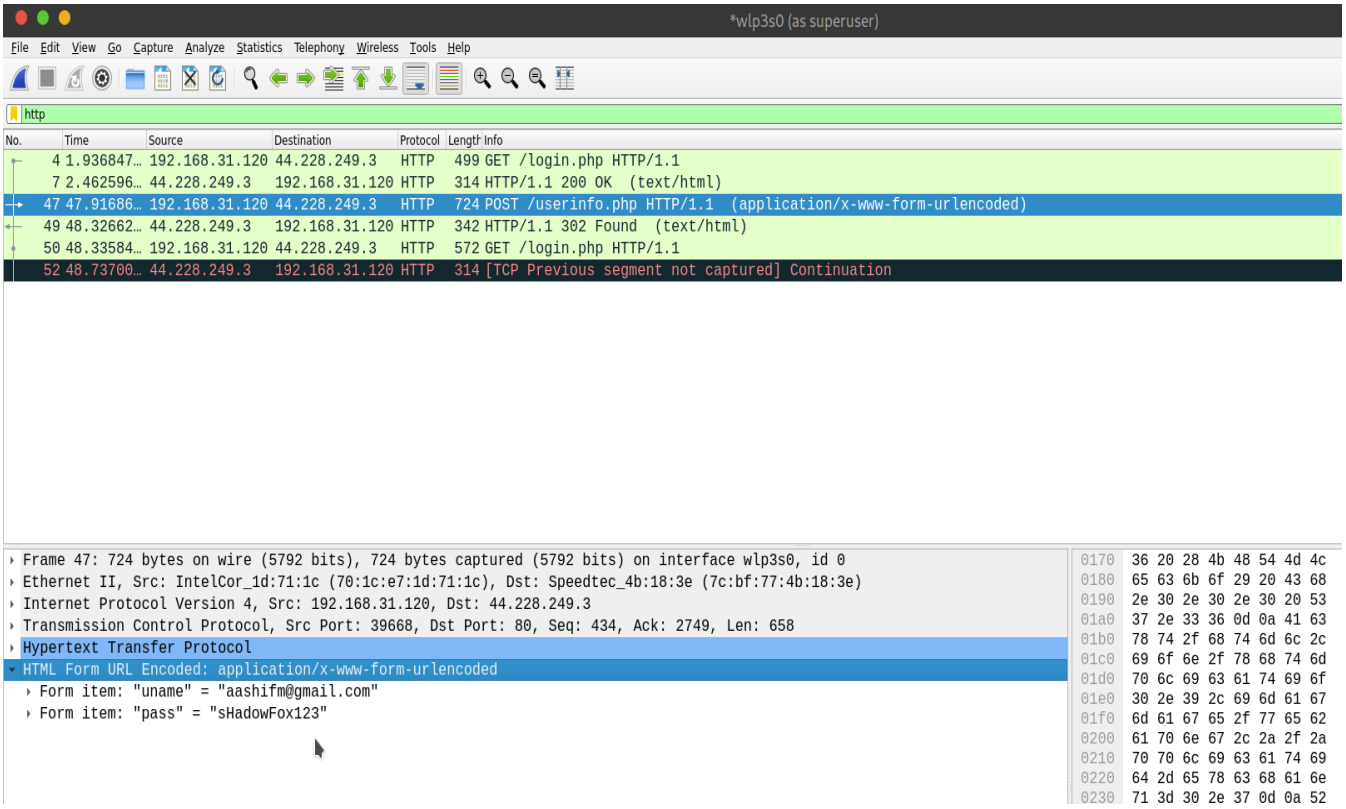
- **Objective: Capture and analyze login credentials transmitted over the network.**



- **Tool Used: Wireshark**
  - **Captured HTTP POST traffic during login to /userinfo.php.**



  - **Filtered and inspected packets to extract credentials sent in plain-text as the site lacks HTTPS encryption.**



- Exposed Credentials:
  - Username: aashifm@gmail.com
  - Password: sHadowFox123
- Risk Noted: Transmitting credentials over an unencrypted channel is a significant security flaw, exposing users to credential theft.

Intermediate Level Tasks

1. Password Cracking for Encrypted Files

- Objective: Crack the password of an MD5-hashed string using a standard wordlist, I followed a systematic approach combining commonly used wordlists with a powerful password recovery tool:

Step 1: Prepare the Wordlist

The widely used `rockyou.txt` wordlist, containing millions of common and leaked passwords, was used for this task.
As the file is often compressed to save space, I decompressed it with:
`sudo gzip -d /usr/share/wordlists/rockyou.txt.gz`
I then verified that the uncompressed file existed at `/usr/share/wordlists/rockyou.txt`.

Step 2: Crack the Hash with Hashcat

I used Hashcat, an industry-standard and highly efficient password cracker.
The command employed was:

```
hashcat -m 0 -a 0 encoded.txt /usr/share/wordlists/rockyou.txt
```

Where:

`-m 0` specifies MD5 hashing.
`-a 0` is for dictionary attack mode.

encoded.txt is the file containing the target hash.
/usr/share/wordlists/rockyou.txt is the dictionary wordlist.

```
Parrot Terminal
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344392
* Bytes.....: 139921507
* Keyspace..: 14344385
* Runtime...: 1 sec

482c811da5d5b4bc6d497ffa98491e38:password123


Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 0 (MD5)
Hash.Target......: 482c811da5d5b4bc6d497ffa98491e38
Time.Started.....: Thu Jul 10 16:10:41 2025 (0 secs)
Time.Estimated...: Thu Jul 10 16:10:41 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.......: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#1.........:    30288 H/s (0.15ms) @ Accel:512 Loops:1 Thr:1 Vec:8
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 2048/14344385 (0.01%)
Rejected.........: 0/2048 (0.00%)
Restore.Point....: 0/14344385 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: 123456 -> lovers1
Hardware.Mon.#1..: Temp: 58c Util: 25%

Started: Thu Jul 10 16:10:07 2025
Stopped: Thu Jul 10 16:10:43 2025

aashifm@parrot ~/Shadowfox/Task-2 $
```

Step 3: Result

After running the command, Hashcat matched the hash to the password password123, demonstrating the effectiveness of dictionary attacks with common wordlists against weak password hashes

2. File Decryption and Secret Extraction

Objective: Decrypt a VeraCrypt-Encrypted File and Extract the Contained Secret

Process:

To access a hidden secret within an encrypted file, I used VeraCrypt, a widely recognized open-source utility for disk encryption. Here's a step-by-step account of the procedure:

Installation and Setup:

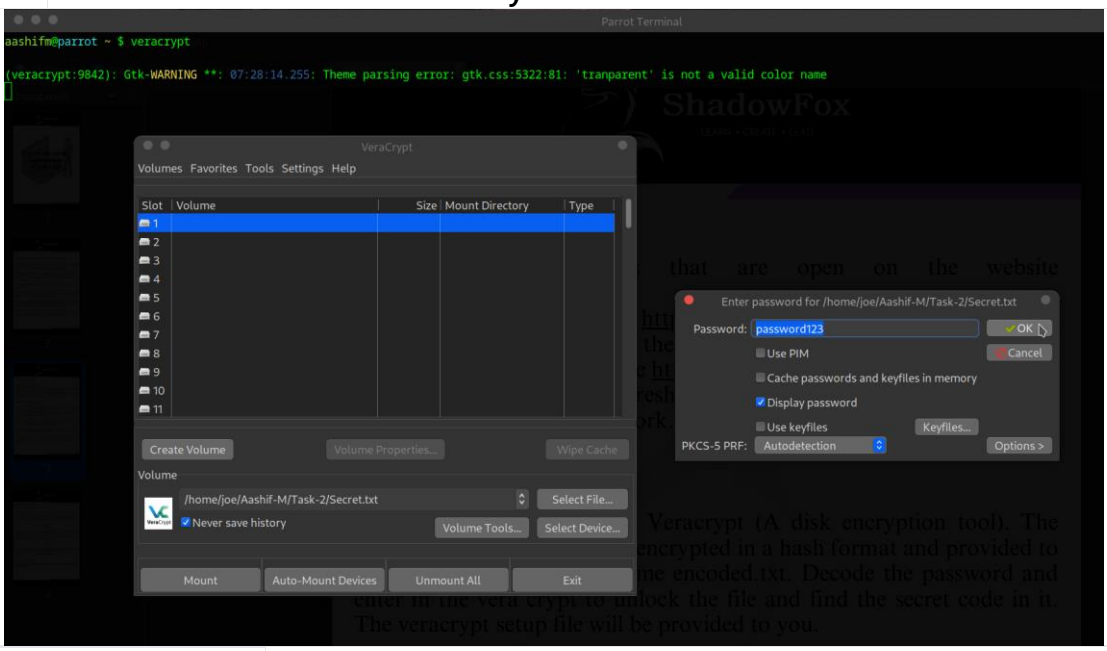
- I began by downloading the appropriate VeraCrypt setup file for my operating system from the [official VeraCrypt website](#).
- The downloaded file (veracrypt-1.26.24-setup.tar.bz2) was extracted using the command:

```
tar -xvjf veracrypt-1.26.24-setup.tar.bz2
```

- I launched the installer (./veracrypt-1.26.24-setup-gui-x64), accepted the license agreement, and completed the installation.

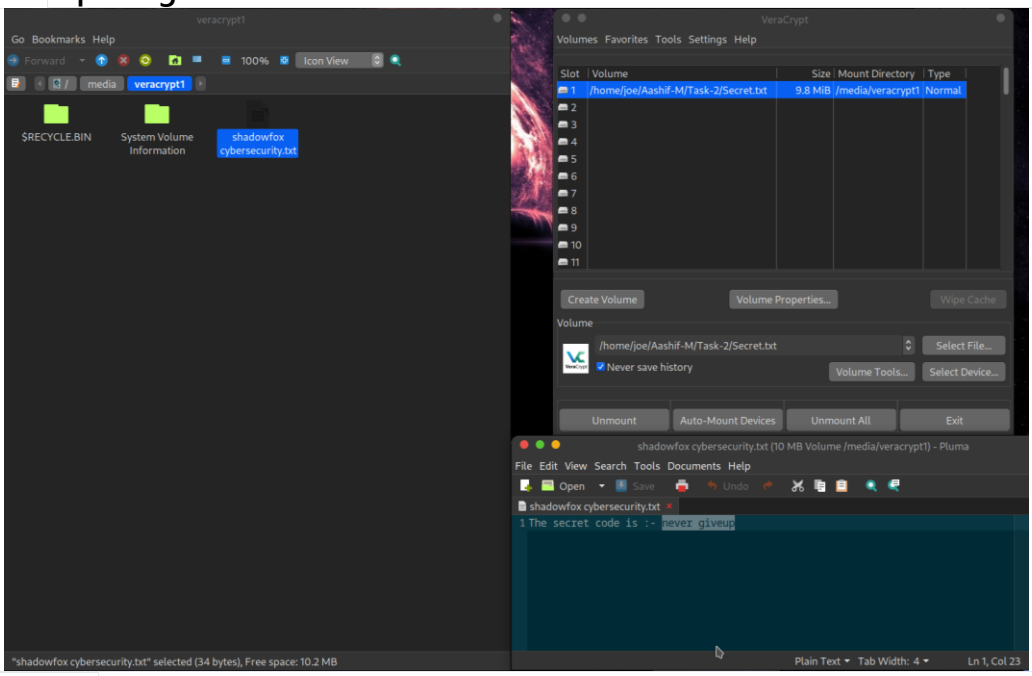
Mounting the Encrypted Volume:

With VeraCrypt installed, I opened the application and mounted the provided encrypted file (in this case labeled as Secret.txt).
I used the previously cracked password (password123) from the MD5 hash task to unlock the volume.
VeraCrypt successfully mounted the encrypted volume, making its contents accessible like a standard drive or directory.



Extracting the Secret:

Navigating to the mounted volume, I found the file named Shadowfox-Cybersecurity.txt.
Opening this file revealed the secret code embedded within.

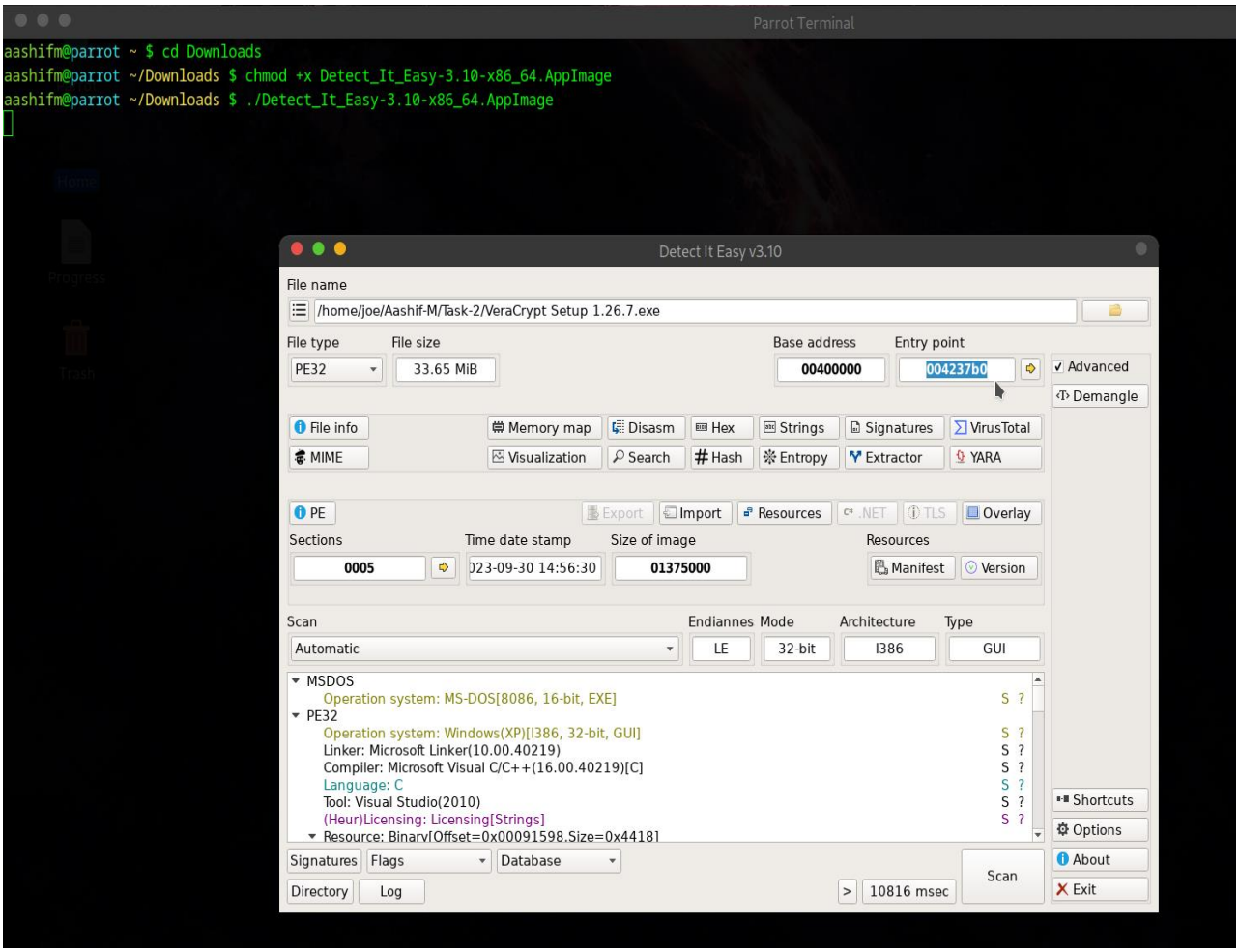


Outcome:

The decrypted file contained the Secret Code: never giveup.

3. PE Entry Point Analysis

- Objective: Identify the entry point address in Veracrypt.exe using PE analysis tools.



- Tool Used: Detect-It-Easy (DIE)
- Granted executable permission, ran the application, and scanned the executable for entry point metadata.

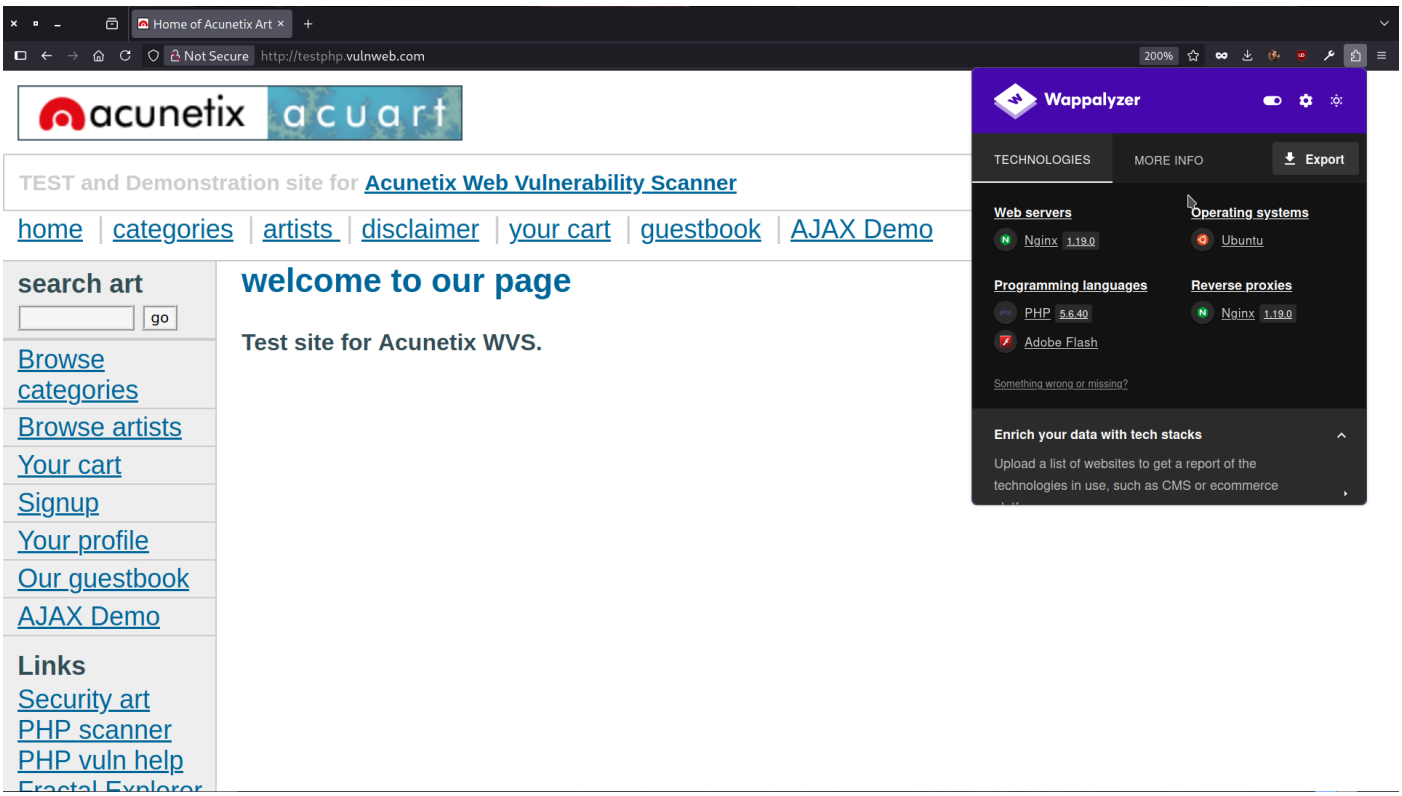
- Entry Point: 004237b0

-
- This address indicates where the program's execution begins when loaded into memory. Significance: Knowing the entry point aids reverse engineering, malware analysis, and advanced binary exploitation.
- Additionally, it can help analysts map execution flow and detect potential vulnerabilities in the program's startup routines. Furthermore, security testers may use the entry point as a reference for identifying injected or modified code regions during forensic investigations.

Hard Level Tasks

1. Technology Stack Identification

- Tool Used: Wappalyzer

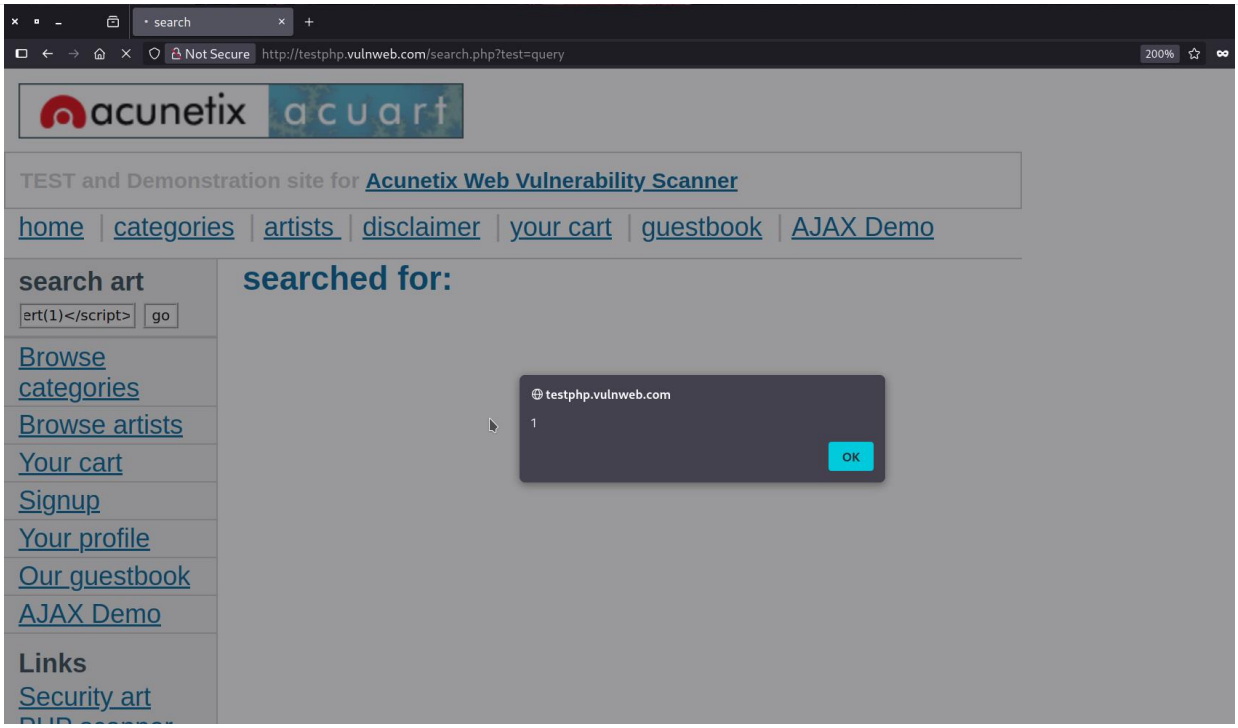


- Findings:
  - Web Server: Nginx 1.19.0
  - OS: Ubuntu
  - Programming Languages: PHP, Adobe Flash
  - Reverse Proxy: Nginx

2. Web Vulnerability Exploitation

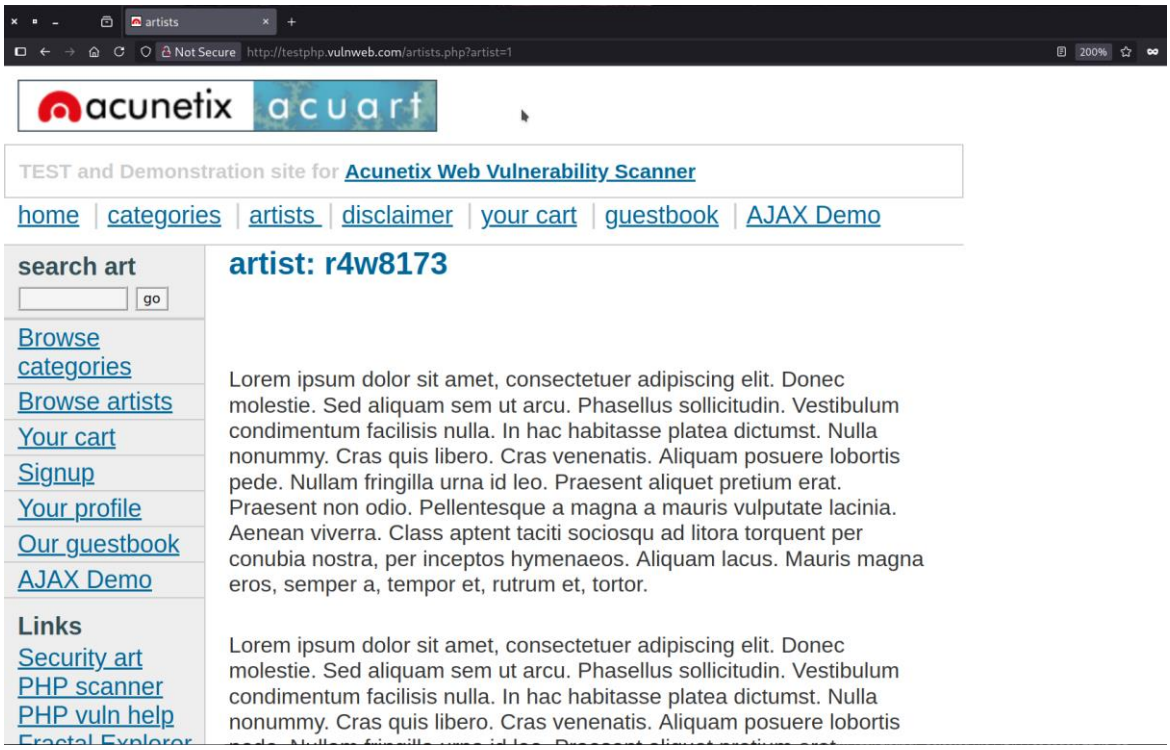
XSS (Cross-Site Scripting)

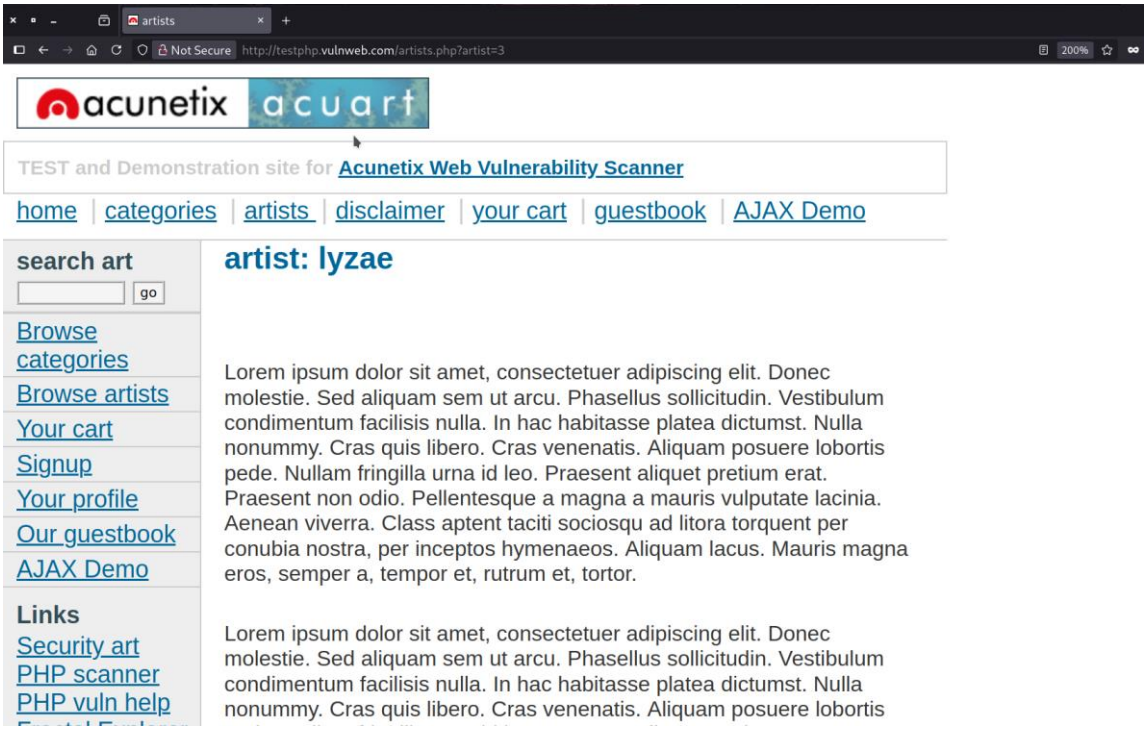
I tested the web application's input fields by injecting a basic script payload: `<script>alert(1)</script>`. The script was executed in the browser, confirming that user input is not properly sanitized or encoded on output. This vulnerability is considered high risk as it allows attackers to execute arbitrary JavaScript in the context of users' browsers, potentially leading to session hijacking, credential theft, or further exploitation.



IDOR (Insecure Direct Object Reference)

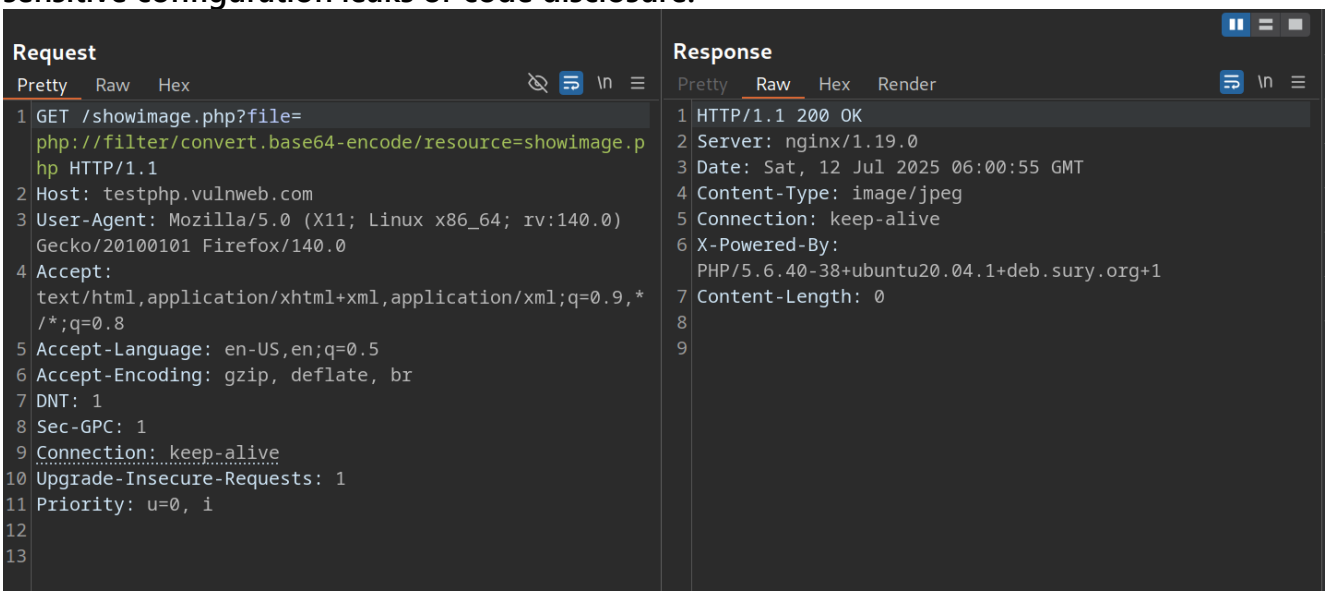
By manipulating predictable parameter values in the endpoint `artists.php` (e.g., changing `artist=1` to `artist=3`), I was able to access detailed information of other users without proper authorization. This confirms the presence of IDOR vulnerabilities, which threaten user privacy and data confidentiality by exposing sensitive records to unauthorized parties.





LFI (Local File Inclusion)

To test for LFI, I used the payload `showimage.php?file=php://filter/convert.base64-encode/resource=showimage.php`. The response demonstrated that server-side files could be read and disclosed by exploiting poorly validated file parameters. This particular use of the PHP filter wrapper allowed viewing the file's content in a base64-encoded format, heightening the risk of sensitive configuration leaks or code disclosure.



Request

Pretty Raw Hex

```
1 GET /showimage.php?file=
  php://filter/convert.base64-encode/resource=showimage.p
  hp HTTP/1.1
2 Host: testphp.vulnweb.com
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0)
  Gecko/20100101 Firefox/140.0
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,*
  /*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 DNT: 1
8 Sec-GPC: 1
9 Connection: keep-alive
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
12
13
```

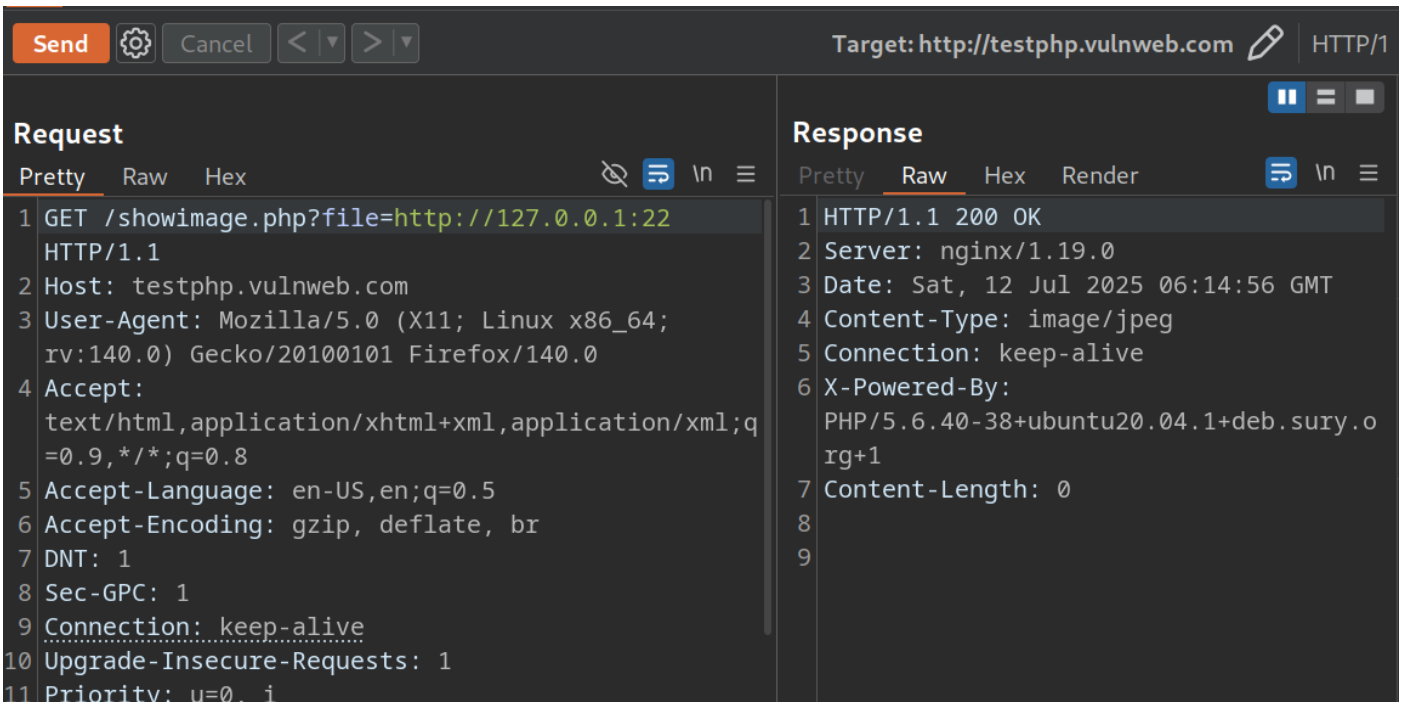
Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Server: nginx/1.19.0
3 Date: Sat, 12 Jul 2025 06:00:55 GMT
4 Content-Type: image/jpeg
5 Connection: keep-alive
6 X-Powered-By:
  PHP/5.6.40-38+ubuntu20.04.1+deb.sury.org+1
7 Content-Length: 0
8
9
```

SSRF (Server-Side Request Forgery)

On the endpoint `showimage.php?file=http://127.0.0.1:22`, I was able to induce the server to make requests to an internal address. This SSRF vulnerability could allow attackers to interact with internal services, potentially retrieving restricted resources or exploiting other services not directly accessible from outside the network.



- Other Notable Findings:
  - Multiple SQL injection points and absence of secure transport (SSL/TLS) detected[2].
  - Sessions not properly secured, increasing risk of session hijacking and data loss.

Learning Outcomes

- Deepened practical experience with leading security tools: Nmap, Gobuster, Wireshark, Hashcat, VeraCrypt, Detect-It-Easy, and Wappalyzer.
- Acquired skills in real-world vulnerability assessment: network reconnaissance, brute-force attacks, traffic interception, password cracking, binary analysis, and advanced web exploitation.
- Enhanced understanding of OWASP Top 10 vulnerabilities and the importance of layered defense.
- Developed systematic documentation and reporting skills for penetration tests.

Conclusion

The Shadowfox cybersecurity internship offered a comprehensive, hands-on experience in modern penetration testing and web application security analysis. It fostered my technical growth and ability to approach real-world security problems using industry-preferred methodologies.